

# COMP 3311

# DATABASE MANAGEMENT

# SYSTEMS

LECTURE 21 EXERCISES

QUERY PROCESSING:

SIZE ESTIMATES; QUERY OPTIMIZATION

# EXERCISE 1

Given the relation Sailor(sailorId, sName, rating, age).

$n_{\text{Sailor}} = 10,000$  tuples

$B_{\text{Sailor}} = 1,000$  pages

$bf_{\text{Sailor}} = \lceil 10,000 / 1,000 \rceil = 10$  tuples/page

$V(\text{rating}, \text{Sailor}) = 10$  (10 distinct rating values)

$V(\text{age}, \text{Sailor}) = 100$  (100 distinct age values)

$SC(\text{rating}=7, \text{Sailor}) = n_{\text{Sailor}} / V(\text{rating}=7, \text{Sailor}) = 10,000 / 10 = \underline{1,000}$  tuples

$SC(\text{age}=40, \text{Sailor}) = n_{\text{Sailor}} / V(\text{age}=40, \text{Sailor}) = 10,000 / 100 = \underline{100}$  tuples

```
select sName
from Sailor
where rating=7
and age=40;
```

Estimate the page I/O cost to process the query using the following alternative plans. Assume uniform distribution and attribute independence.

*Ignore the cost of searching any indexes.*

- a) linear search
- b) binary search
- c) single B<sup>+</sup>-tree index (on either attribute)
- d) multiple B<sup>+</sup>-tree indexes (on both rating and age)

```
select sName
from Sailor
where rating=7
and age=40;
```

## EXERCISE 1 (CONT'D)

$n_{\text{Sailor}}$ : 10,000 tuples  
 $B_{\text{Sailor}}$ : 1,000 pages  
 $bf_{\text{Sailor}}$ : 10 tuples/page  
 $V(\text{rating}, \text{Sailor})$ : 10 distinct values  
 $V(\text{age}, \text{Sailor})$ : 100 distinct values  
 $SC(\text{rating}=7, \text{Sailor})$  = 1,000 tuples  
 $SC(\text{age}=40, \text{Sailor})$  = 100 tuples

### a) linear search page I/O cost

Read the whole **Sailor** relation and select the records that satisfy both conditions.

Page I/O cost:  $B_{\text{Sailor}} = 1,000$

### b) binary search page I/O cost

If the file is sorted on **rating** (or **age**), do a binary search and find the first record satisfying the condition **rating=7** (or **age=40**).

Retrieve the remaining records sequentially and discard tuples that do not meet the other condition.

**Question:** Which is cheaper, searching a file sorted on **rating** or on **age**?

```
select sName
from Sailor
where rating=7
and age=40;
```

## EXERCISE I (CONTD)

$n_{\text{Sailor}}$ : 10,000 tuples  
 $B_{\text{Sailor}}$ : 1,000 pages  
 $bf_{\text{Sailor}}$ : 10 tuples/page  
 $V(\text{rating}, \text{Sailor})$ : 10 distinct values  
 $V(\text{age}, \text{Sailor})$ : 100 distinct values  
 $SC(\text{rating}=7, \text{Sailor}) = 1,000$  tuples  
 $SC(\text{age}=40, \text{Sailor}) = 100$  tuples

i. Search a file sorted on rating

$$\begin{aligned}
 \text{Page I/O cost: } & \overset{\text{\# I/Os to find 1st page}}{\lceil \log_2(B_{\text{Sailor}}) \rceil} + \overset{\text{\# I/Os to retrieve all pages}}{\lceil SC(\text{rating}=7, \text{Sailor}) / bf_{\text{Sailor}} \rceil} - \overset{\text{1st page I/O}}{1} \\
 &= \lceil \log_2(1,000) \rceil + \lceil 1,000 / 10 \rceil - 1 \\
 &= 10 + 99 \\
 &= \underline{109}
 \end{aligned}$$

ii. Search a file sorted on age

$$\begin{aligned}
 \text{Page I/O cost: } & \overset{\text{\# I/Os to find 1st page}}{\lceil \log_2(B_{\text{Sailor}}) \rceil} + \overset{\text{\# I/Os to retrieve all pages}}{\lceil SC(\text{age}=40, \text{Sailor}) / bf_{\text{Sailor}} \rceil} - \overset{\text{1st page I/O}}{1} \\
 &= \lceil \log_2(1,000) \rceil + \lceil 100 / 10 \rceil - 1 \\
 &= 10 + 9 \\
 &= \underline{19}
 \end{aligned}$$

✎ Since there are more distinct age values than rating values, each age will have fewer qualifying tuples  $\Rightarrow$  fewer pages to retrieve for a given age.

```
select sName
from Sailor
where rating=7
and age=40;
```

## EXERCISE 1 (CONTD)

$n_{\text{Sailor}}$ : 10,000 tuples  
 $B_{\text{Sailor}}$ : 1,000 pages  
 $bf_{\text{Sailor}}$ : 10 tuples/page  
 $V(\text{rating}, \text{Sailor})$ : 10 distinct values  
 $V(\text{age}, \text{Sailor})$ : 100 distinct values  
 $SC(\text{rating}=7, \text{Sailor}) = 1,000$  tuples  
 $SC(\text{age}=40, \text{Sailor}) = 100$  tuples

### c) single B<sup>+</sup>-tree index (on either attribute) page I/O cost

Use the index for one of the two conditions and check the other condition *on-the-fly* in the buffer.

✎ **Not necessarily a good solution if the index is not clustered.**

#### i. index on rating only

Use the index to find records where  $\text{rating}=7$ .

For each qualifying record, check the condition  $\text{age}=40$  *on-the-fly* in the buffer.

For  $\text{rating}=7$ ,  $V(\text{rating}, \text{Sailor}) = 10$  distinct values.

$SC(\text{rating}=7, \text{Sailor}) = n_{\text{Sailor}} / V(\text{rating}=7, \text{Sailor}) = 10,000 / 10 = \underline{1,000}$  tuples.

Sailor has 1,000 pages.

If the index on rating is **not clustered**, to retrieve these sailors we need 1,000 random page I/Os.

✎ **Might as well do a sequential scan!**

If the index is **clustered**, all sailors with  $\text{rating}=7$  are in  $\lceil 1,000 / 10 \rceil = \underline{100}$  consecutive pages.

```
select sName
from Sailor
where rating=7
and age=40;
```

## EXERCISE 1 (CONT'D)

$n_{\text{Sailor}}$ : 10,000 tuples  
 $B_{\text{Sailor}}$ : 1,000 pages  
 $bf_{\text{Sailor}}$ : 10 tuples/page  
 $V(\text{rating}, \text{Sailor})$ : 10 distinct values  
 $V(\text{age}, \text{Sailor})$ : 100 distinct values  
 $SC(\text{rating}=7, \text{Sailor}) = 1,000$  tuples  
 $SC(\text{age}=40, \text{Sailor}) = 100$  tuples

### ii. index on age only

Use the index to find records where  $\text{age}=40$ .

For each qualifying record, check the condition  $\text{rating}=7$  *on-the-fly* in the buffer.

For  $\text{age}=40$ ,  $V(\text{age}, \text{Sailor}) = 100$  distinct values.

$SC(\text{age}=40, \text{Sailor}) = n_{\text{Sailor}} / V(\text{age}=40, \text{Sailor}) = 10,000 / 100 = 100$  tuples.

Sailor has 1,000 pages.

If the index on rating is **not clustered**, to retrieve these sailors we need 100 random page I/Os.

👉 **Better than a sequential scan!**

If the index is **clustered**, all sailors with  $\text{age}=40$  are in  $\lceil 100 / 10 \rceil = 10$  consecutive pages.

```
select sName
from Sailor
where rating=7
and age=40;
```

## EXERCISE 1 (CONTD)

$n_{\text{Sailor}}$ : 10,000 tuples  
 $B_{\text{Sailor}}$ : 1,000 pages  
 $bf_{\text{Sailor}}$ : 10 tuples/page  
 $V(\text{rating}, \text{Sailor})$ : 10 distinct values  
 $V(\text{age}, \text{Sailor})$ : 100 distinct values  
 $SC(\text{rating}=7, \text{Sailor}) = 1,000$  tuples  
 $SC(\text{age}=40, \text{Sailor}) = 100$  tuples

d) multiple B<sup>+</sup>-tree indexes (on both rating and age)

Use more than one index and **intersect record pointers** (rids) before retrieval of actual tuples.

Use the **index on age** to find all rids of tuples where **age=40**.

$$\text{Selectivity}(\text{age}=40) = SC(\text{age}=40, \text{Sailor}) / n_{\text{Sailor}} = 100 / 10,000 = 0.01.$$

Use the **index on rating** to find all rids of tuples where **rating=7**.

$$\text{Selectivity}(\text{rating}=7) = SC(\text{rating}=7, \text{Sailor}) / n_{\text{Sailor}} = 1,000 / 10,000 = 0.1.$$

$$\begin{aligned} \text{Estimated size: } n_{\text{Sailor}} * \text{selectivity}(\text{age}=40) * \text{selectivity}(\text{rating}=7) \\ = 10,000 * 0.01 * 0.1 = \underline{10} \text{ tuples.} \end{aligned}$$

Page I/O cost: 10 (as each tuple could be on a different page)

### Page I/O cost summary

| <u>Linear search</u> | <u>Binary search</u> |                | <u>Single Clustering B<sup>+</sup>-tree</u> |        | <u>Multiple B<sup>+</sup>-trees</u> |
|----------------------|----------------------|----------------|---------------------------------------------|--------|-------------------------------------|
|                      | ordered on rating    | ordered on age | on rating                                   | on age | on rating and age                   |
| 1,000                | 109                  | 19             | 1,000                                       | 100    | 10                                  |

## EXERCISE 2

Given the following relations and the information about them, estimate the **minimum page I/O cost** to process the query shown on the right **using materialization** and the join order (Sailor JOIN  $\sigma_{rDate='01-JAN-2025'}$  Reserves) JOIN  $\sigma_{color='red'}$  Boat.

```
select *  
from Sailor natural join Reserves natural join Boat  
where rDate = '01-JAN-2025'  
and color = 'red';
```

Sailor(sailorId: 2 bytes, sName: 10 bytes, hkid: 10 bytes, rating: 2 bytes, age: 2 bytes)

Reserves(sailorId: 2 bytes, boatId: 2 bytes, rDate: 7 bytes)

Boat(boatId: 2 bytes, bName: 10 bytes, color: 10 bytes, bType: 10 bytes)

There are 10,000 Sailor tuples, 100,000 Reserves tuples and 1,000 Boat tuples.

$bf_{Sailor}$ : 10

$bf_{Reserves}$ : 23

$bf_{Boat}$ : 8

$B_{Sailor}$ : 1,000 pages

$B_{Reserves}$ : 4,348 pages

$B_{Boat}$ : 125 pages

The page size is 260 bytes, and the buffer size  $M$  is 100 pages.

There are the following indexes:

**hash index** on sailorId for Sailor  
(no overflow buckets).

**hash index** on boatId for Boat  
(no overflow buckets).

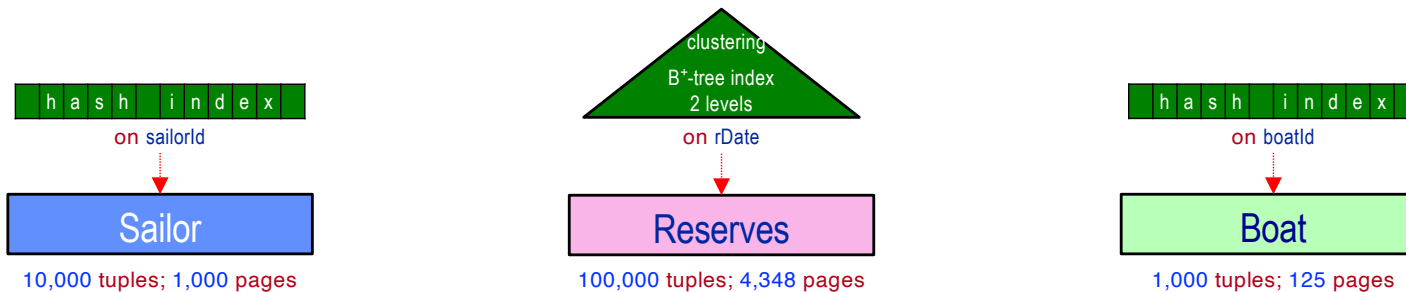
**clustering B<sup>+</sup>-tree index** on rDate for Reserves  
(2 levels).

### Some useful statistics:

- Reserves has 1,000 unique dates.
- 10% of boats are red.
- A sailor has on average 10 reservations.



## EXERCISE 2 (CONT'D)



```
select *
from Sailor natural join Reserves natural join Boat
where rDate = '01-JAN-2025'
and color = 'red';
```

### Some useful statistics:

- Reserves has 1,000 unique dates.
- 10% of boats are red.
- A sailor has on average 10 reservations

The cost to compute  $(\text{Sailor JOIN } \sigma_{\text{rDate}='01\text{-JAN-2025'}}\text{Reserves}) \text{ JOIN } \sigma_{\text{color}='red'}\text{Boat}$  is

$C_1$ : Cost to compute  $\text{TempC}_1 = (\text{Sailor JOIN } \sigma_{\text{rDate}='01\text{-JAN-2025'}}\text{Reserves})$

$C_2$ : Cost to compute  $\text{TempC}_2 = \sigma_{\text{color}='red'}\text{Boat}$  (no index on color)

$C_3$ : Cost to compute  $\text{TempC}_1 \text{ JOIN } \text{TempC}_2$

Query processing page I/O cost:  $C_1 + C_2 + C_3$

## EXERCISE 2 (CONTD)

$C_1$ : Cost to compute  $\text{Temp}C_1 = (\text{Sailor JOIN } \sigma_{\text{rDate}='01\text{-JAN-2025'}} \text{Reserves})$

To estimate  $C_1$ , the best evaluation plan (sub-plan) for computing  $\text{Temp}_1$  needs to be determined.

Some alternative join strategies:

$C_{11}$ : optimized block nested-loop (outer relation Sailor):  $\lceil B_r / (M-2) \rceil * B_s + B_r$ .

$C_{12}$ : optimized block nested-loop (outer relation Reserves):  $\lceil B_r / (M-2) \rceil * B_s + B_r$ .

$C_{13}$ : sort-merge join:  $B_r + B_s + \text{sorting cost: } 2 * B_r * (1 + \lceil \log_{M-1}(B_r/M) \rceil)$ .

$C_{14}$ : indexed nested-loop (outer relation Reserves):  $B_r + n_r * c$ .

Note: For strategies  $C_{11}$  to  $C_{13}$ , the selection  $\sigma_{\text{rDate}='01\text{-JAN-2025'}}$  is done *on-the-fly* in the buffer as the join is computed.

For strategy  $C_{14}$  the selection condition  $\text{rDate}='01\text{-JAN-2025'}$  is used to perform an index lookup on Reserves before using indexed nested-loop to join with Sailor.

```
select *
from Sailor natural join Reserves natural join Boat
where rDate = '01-JAN-2025'
and color = 'red';
```

## EXERCISE 2 (CONTD)

*bf*<sub>Sailor</sub>: 10 tuples/page  
*bf*<sub>Reserves</sub>: 23 tuples/page  
*bf*<sub>Boat</sub>: 8 tuples/page  
*B*<sub>Sailor</sub>: 1,000 pages  
*B*<sub>Reserves</sub>: 4,348 pages  
*B*<sub>Boat</sub>: 125 pages  
 Buffer pages *M*: 100  
 Page size: 260  
 Hash index: Sailor.sailorId  
 B\*-tree index: Reserves.rDate  
 (2 levels)  
 Hash index: Boat.boatId  
 Unique Reserves rDates: 1000  
 Red boats: 10% (100 tuples)  
 Reservations/sailor: 10

**C<sub>1</sub>**: Cost to compute TempC<sub>1</sub> = (Sailor JOIN  $\sigma_{rDate='01-JAN-2025'}$  Reserves)

Strategy C<sub>1</sub>: optimized block nested-loop (outer relation Sailor).

Query page I/O cost:  $\lceil B_r / (M-2) \rceil * B_s + B_r$   
 $= \lceil 1000 / (100-2) \rceil * 4348 + 1000 = \underline{48,828}$

Strategy C<sub>1</sub>2: optimized block nested-loop (outer relation Reserves).

Query page I/O cost:  $\lceil B_r / (M-2) \rceil * B_s + B_r$   
 $= \lceil 4348 / (100-2) \rceil * 1000 + 4348 = \underline{49,348}$

Strategy C<sub>13</sub>: sort-merge join:  $B_r + B_s$  + sorting cost.

Sort Sailor page I/O cost:  $2 * B_r * (1 + \lceil \log_{M-1}(B_r/M) \rceil)$   
 $= 2 * 1000 * (1 + \lceil \log_{100-1}(1000/100) \rceil) = 4,000$

Sort Reserves page I/O cost:  $2 * B_r * (1 + \lceil \log_{M-1}(B_r/M) \rceil)$   
 $= 2 * 4348 * (1 + \lceil \log_{100-1}(4348/100) \rceil) = 17,392$

Merge page I/O cost:  $B_r + B_s = 1000 + 4348 = 5,348$

Query page I/O cost: 4000 + 17392 + 5348 = 26,740

```

select *
from Sailor natural join Reserves natural join Boat
where rDate = '01-JAN-2025'
and color = 'red';

```

## EXERCISE 2 (CONTD)

$bf_{Sailor}$ : 10 tuples/page  
 $bf_{Reserves}$ : 23 tuples/page  
 $bf_{Boat}$ : 8 tuples/page  
 $B_{Sailor}$ : 1,000 pages  
 $B_{Reserves}$ : 4,348 pages  
 $B_{Boat}$ : 125 pages  
 Buffer pages  $M$ : 100  
 Page size: 260  
 Hash index: Sailor.sailorId  
 B<sup>+</sup>-tree index: Reserves.rDate  
 (2 levels)  
 Hash index: Boat.boatId  
 Unique Reserves rDates: 1000  
 Red boats: 10% (100 tuples)  
 Reservations/sailor: 10

**Strategy C<sub>14</sub>**: index lookup using B<sup>+</sup>-tree index on rDate and indexed nested-loop using hash index on sailorId

There is a selective condition ( $\sigma_{rDate='01-JAN-2025'}(Reserves)$ ) and a clustering index on Reserves.rDate. Moreover, Sailor has a hash index on the join attribute sailorId.

How many reservations need to be accessed?

Since there are 100,000 reservations and 1,000 unique rDates, on a given rDate there are  $100000 / 1000 = 100$  reservations. Therefore, 100 Reserves tuples need to be accessed for 01-JAN-2025.

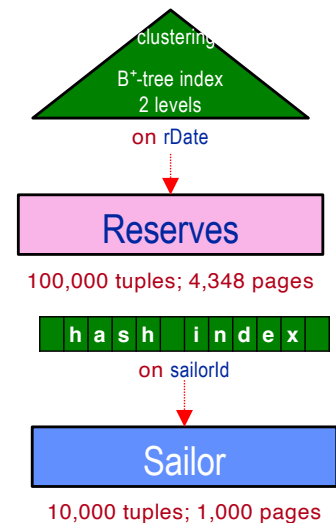
How many index pages need to be accessed, to find these 100 reservations?

The index on Reserves.rDate has 2 levels and is ordered on rDate.

Since,  $bf_{Reserves} = 23$ , to retrieve 100 reservations 2 index pages and  $\lceil 100/23 \rceil = 5$  Reserves pages = 7 page I/Os are needed.

Finally, join each of the 100 Reserves tuples with Sailor using the hash index on Sailor.sailorId, requiring  $2 * 100 = 200$  page I/Os.

Join page I/O cost:  $7 + 200 = 207$



```

select *
from Sailor natural join Reserves natural join Boat
where rDate = '01-JAN-2025'
and color = 'red';

```

## EXERCISE 2 (CONTD)

$bf_{Sailor}$ : 10 tuples/page  
 $bf_{Reserves}$ : 23 tuples/page  
 $bf_{Boat}$ : 8 tuples/page  
 $B_{Sailor}$ : 1,000 pages  
 $B_{Reserves}$ : 4,348 pages  
 $B_{Boat}$ : 125 pages  
 Buffer pages  $M$ : 100  
 Page size: 260  
 Hash index: Sailor.sailorId  
 B<sup>+</sup>-tree index: Reserves.rDate  
 (2 levels)  
 Hash index: Boat.boatId  
 Unique Reserves rDates: 1000  
 Red boats: 10% (100 tuples)  
 Reservations/sailor: 10

### Cost summary for computing TempC<sub>1</sub>

#### Strategy C<sub>1</sub>1

optimized block  
nested-loop join  
(Sailor outer relation)

48,828

#### Strategy C<sub>1</sub>2

optimized block nested-  
loop join  
(Reserves outer relation)

49,348

#### Strategy C<sub>1</sub>3

sort-merge join

26,740

#### Strategy C<sub>1</sub>4

indexed nested-loop  
join

207

For Strategies C<sub>1</sub>1 to C<sub>1</sub>3, the condition rDate='01-JAN-2025' is checked in the buffer.  
For Strategy C<sub>1</sub>4 the condition rDate='01-JAN-2025' is used to perform an index lookup on Reserves before joining with Sailor.

The join result will contain 100 tuples, since each Reserves tuple will join with exactly one Sailor tuple,

The join result size  $(26+11-2)=35$  bytes. Therefore,  $bf_{joinresult} = \lfloor 260/35 \rfloor = 7$ .

Thus, TempC<sub>1</sub> will occupy  $\lceil 100/7 \rceil = 15$  pages.

Page I/O cost to write TempC<sub>1</sub>: 15

Total page I/O cost for best C<sub>1</sub> strategy that minimizes overall page I/O cost  
(C<sub>1</sub>4):  $\underset{r}{207} + \underset{w}{15} = \underline{222}$

```
select *
from Sailor natural join Reserves natural join Boat
where rDate = '01-JAN-2025'
and color = 'red';
```

## EXERCISE 2 (CONTD)

*bf*<sub>Sailor</sub>: 10 tuples/page  
*bf*<sub>Reserves</sub>: 23 tuples/page  
*bf*<sub>Boat</sub>: 8 tuples/page  
*B*<sub>Sailor</sub>: 1,000 pages  
*B*<sub>Reserves</sub>: 4,348 pages  
*B*<sub>Boat</sub>: 125 pages  
 Buffer pages *M*: 100  
 Page size: 260  
 Hash index: Sailor.sailorId  
 B\*-tree index: Reserves.rDate  
                     (2 levels)  
 Hash index: Boat.boatId  
 Unique Reserves rDates: 1000  
 Red boats: 10% (100 tuples)  
 Reservations/sailor: 10

$C_2$ : Cost of computing  $\text{Temp}C_2 = \sigma_{\text{color}='red'} \text{Boat}$  (*no index on color*)

## Strategy C<sub>2</sub>1: linear search on Boat

Read 125 Boat pages. Discard tuples where the color is not red.

Page I/O cost for selection: 125

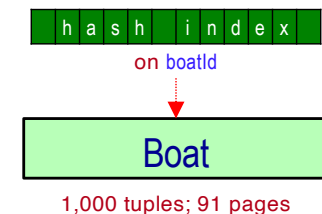
Since 10% of boats are red and there are 1,000 Boat tuples, it is expected that  $10\% \cdot 1000 = 100$  Boat tuples are selected, which fit in  $\lceil 100 / 8 \rceil = \lceil 12.5 \rceil = 13$  pages.

Page I/O cost to write TempC<sub>2</sub>1: 13

Total page I/O cost for Strategy C<sub>2</sub>1:  $\underset{r}{125} + \underset{w}{13} = \underline{138}$

## Strategy C<sub>2</sub>: use hash index on Boat.boatId

For each of the 100 TempC<sub>1</sub> tuples hash the boatId to access the hash index and retrieve the corresponding Boat tuple. Discard tuples where the color is not red. The cost is 2 page I/Os for each TempC<sub>1</sub> tuple (1 for the hash index and 1 for the Boat tuple).



Page I/O cost to read TempC<sub>1</sub>: 15

Page I/O cost to use hash index on Boat:  $2 * 100 = 200$

Page I/O cost for selection:  $15 + 200 = 215$

Since 10% of boats are red, it is expected that  $\lceil 10\% * 100 \rceil = 10$  Boat tuples are in the result, which fit in  $\lceil 10 / 8 \rceil = 2$  pages.

```
select *
from Sailor natural join Reserves natural join Boat
where rDate = '01-JAN-2025'
and color = 'red';
```

## EXERCISE 2 (CONTD)

*bf*<sub>Sailor</sub>: 10 tuples/page  
*bf*<sub>Reserves</sub>: 23 tuples/page  
*bf*<sub>Boat</sub>: 8 tuples/page  
*B*<sub>Sailor</sub>: 1,000 pages  
*B*<sub>Reserves</sub>: 4,348 pages  
*B*<sub>Boat</sub>: 125 pages  
 Buffer pages *M*: 100  
 Page size: 260  
 Hash index: Sailor.sailorId  
 B\*-tree index: Reserves.rDate  
 (2 levels)  
 Hash index: Boat.boatId  
 Unique Reserves rDates: 1000  
 Red boats: 10% (100 tuples)  
 Reservations/sailor: 10

Page I/O cost to write TempC<sub>2</sub>: 2

Total page I/O cost for Strategy C<sub>2</sub>2: 215 + 2 = 217

Page I/O cost for best  $C_2$  strategy that minimizes overall page I/O cost ( $C_21$ ): 138

$C_3$ : Cost of TempC<sub>1</sub> JOIN TempC<sub>2</sub>

Strategy C<sub>3</sub>1: block nested-loop join after Strategy C<sub>2</sub>1 (linear search on Boat).

Since both  $\text{TempC}_1$  (15 pages) and  $\text{TempC}_2$  (13 pages) fit in the buffer, the join cost is  $B_{\text{TempC1}} + B_{\text{TempC2}}$ .

Page I/O cost to read TempC<sub>1</sub>: 15      Page I/O cost to read TempC<sub>2</sub>: 13

Total page I/O cost for Strategy C<sub>3</sub>:  $\underset{r}{15} + \underset{r}{13} = \underline{28}$

Strategy C<sub>32</sub>: block nested-loop join after Strategy C<sub>22</sub> (hash index on Boat.boatId).

Since both  $\text{TempC}_1$  (15 pages) and  $\text{TempC}_2$  (2 pages) fit in the buffer, the join cost is  $B_{\text{TempC1}} + B_{\text{TempC2}}$ .

Page I/O cost to read TempC<sub>1</sub>: 15      Page I/O cost to read TempC<sub>2</sub>: 2

Total page I/O cost for Strategy C<sub>3</sub>2:  $\underbrace{15}_r + \underbrace{2}_r = \underline{17}$

Page I/O cost for best  $C_3$  strategy that minimizes overall page I/O cost ( $C_31$ ): 28

## EXERCISE 2 (CONTD)

### Minimum query processing page I/O cost

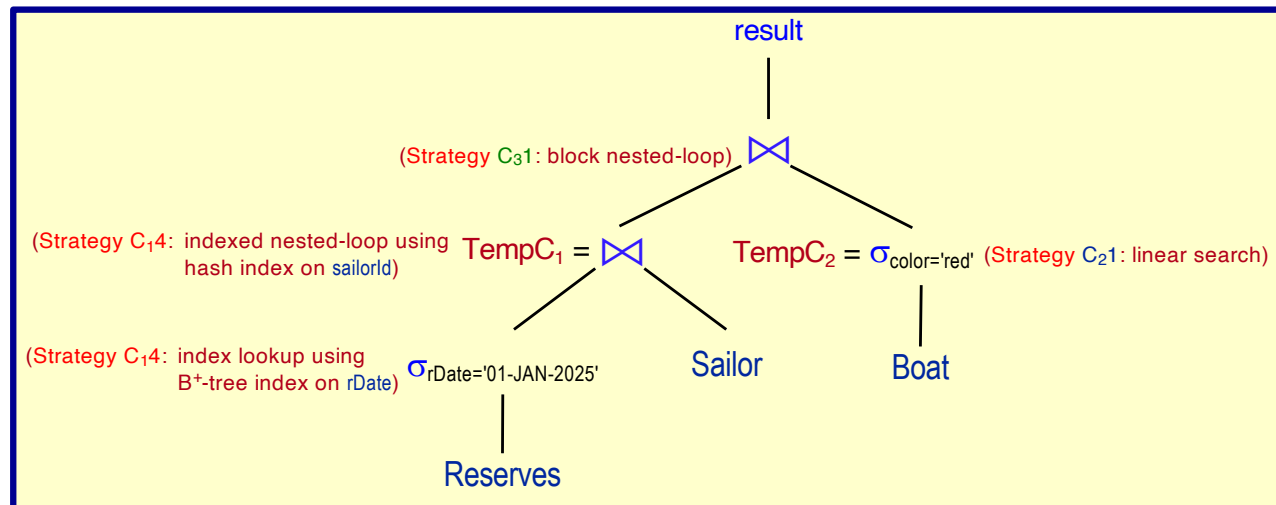
|                                                |            |
|------------------------------------------------|------------|
| $C_14$ : index lookup &<br>indexed nested loop | = 222      |
| $C_21$ : linear search                         | = 138      |
| $C_31$ : block nested-loop                     | = 28       |
| Total                                          | <u>388</u> |

### Alternate query processing page I/O cost

|                                                |            |
|------------------------------------------------|------------|
| $C_14$ : index lookup &<br>indexed nested-loop | = 222      |
| $C_22$ : hash index                            | = 217      |
| $C_32$ : block nested-loop                     | = 17       |
| Total                                          | <u>456</u> |

**Recall:** The cost to write the query result is never included in the cost estimate.

### Minimum Query Processing Page I/O Cost Query Evaluation Tree





## EXERCISE 3

Using the information given for Exercise 2, estimate the **minimum page I/O cost** to process the query using materialization and the join order

Sailor JOIN ( $\sigma_{rDate='01-JAN-2025'}$ Reserves JOIN  $\sigma_{color='red'}$ Boat)

$C_1$ : Cost of computing  $TempC_1 = \sigma_{rDate='01-JAN-2025'}$ Reserves

$C_2$ : Cost of computing  $TempC_2 = TempC_1$  JOIN  $\sigma_{color='red'}$ Boat (no index on color)

$C_3$ : Cost of Sailor JOIN  $TempC_2$

Query processing page I/O cost:  $C_1 + C_2 + C_3$

```
select *  
from Sailor natural join Reserves natural join Boat  
where rDate = '01-JAN-2025'  
and color = 'red';
```

```

select *
from Sailor natural join Reserves natural join Boat
where rDate = '01-JAN-2025'
and color = 'red';

```

## EXERCISE 3 (CONT'D)

$bf_{Sailor}$ : 10 tuples/page  
 $bf_{Reserves}$ : 23 tuples/page  
 $bf_{Boat}$ : 8 tuples/page  
 $B_{Sailor}$ : 1,000 pages  
 $B_{Reserves}$ : 4,348 pages  
 $B_{Boat}$ : 125 pages  
 Buffer pages  $M$ : 100  
 Page size: 260  
 Hash index: Sailor.sailorId  
 B<sup>+</sup>-tree index: Reserves.rDate  
 (2 levels)  
 Hash index: Boat.boatId  
 Unique Reserves rDates: 1000  
 Red boats: 10% (100 tuples)  
 Reservations/sailor: 10

$C_1$ : Cost of computing  $TempC_1 = \sigma_{rDate='01-JAN-2025'} Reserves$

Strategy  $C_11$ : linear search on Reserves

Read 4,348 Reserves pages; discard tuples where  $rDate \neq 01-JAN-2025$ .

Page I/O cost for selection: 4,348

For a given  $rDate$  there are  $\lceil 10,000 / 1000 \rceil = 100$  Reserves tuples which fit in  $\lceil 100 / 23 \rceil = 5$  pages.

Page I/O cost to write  $TempC_1$ : 5

Total page I/O cost for Strategy  $C_11$ :  $4,348 + 5 = \underline{4,353}$

**Best  
overall  
strategy**

Strategy  $C_12$ : index lookup using B<sup>+</sup>-tree index on Reserves.rDate

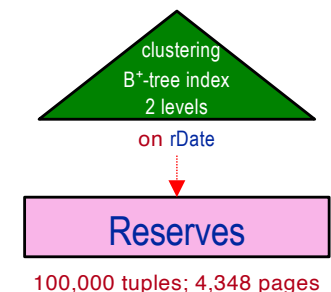
As determined in Exercise 2, to find all reservations on 01-JAN-2025 2 index pages are accessed, and 100 Reserves tuples are retrieved which occupy  $\lceil 100 / 23 \rceil = 5$  pages in the Reserves relation (recall that Reserves is ordered on  $rDate$ ).

Page I/O cost for selection:  $2 + 5 = \underline{7}$

The 100 Reserves tuples (5 pages) are written out as  $TempC_1$ .

Page I/O cost to write  $TempC_1$ : 5

Total page I/O cost for Strategy  $C_12$ :  $7 + 5 = \underline{12}$



```

select *
from Sailor natural join Reserves natural join Boat
where rDate = '01-JAN-2025'
and color = 'red';

```

## EXERCISE 3 (CONT'D)

$bf_{Sailor}$ : 10 tuples/page  
 $bf_{Reserves}$ : 23 tuples/page  
 $bf_{Boat}$ : 8 tuples/page  
 $B_{Sailor}$ : 1,000 pages  
 $B_{Reserves}$ : 4,348 pages  
 $B_{Boat}$ : 125 pages  
 Buffer pages  $M$ : 100  
 Page size: 260  
 Hash index: Sailor.sailorId  
 B<sup>+</sup>-tree index: Reserves.rDate (2 levels)  
 Hash index: Boat.boatId  
 Unique Reserves rDates: 1000  
 Red boats: 10% (100 tuples)  
 Reservations/sailor: 10

$C_2$ : Cost to compute  $TempC_2 = TempC_1 \text{ JOIN } \sigma_{color='red'} Boat$  (no index on color)

**Best overall strategy**

Strategy  $C_21$ : block nested-loop join

Read 5 pages of  $TempC_1$  containing 100 Reserves tuples which fit in the buffer.

Page I/O cost to read  $TempC_1$ : 5

Read 125 pages of Boat and join with  $TempC_1$  in the buffer. Discard tuples where the color is not red.

Page I/O cost to read Boat: 125

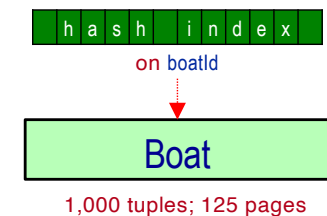
Strategy  $C_22$ : indexed nested-loop join using hash index on Boat.boatId

Read 5 pages of  $TempC_1$  containing 100 Reserves tuples which fit in the buffer.

Page I/O cost to read  $TempC_1$ : 5

For each of the 100  $TempC_1$  Reserves tuples, use its boatId to access the Boat hash index on boatId to find the corresponding Boat tuple. Discard tuples where the color is not red. The cost is 2 page I/Os (1 to the hash index and 1 to access the Boat tuple) per  $TempC_1$  tuple.

Page I/O cost to use hash index on Boat:  $2 * 100 = 200$



```
select *
from Sailor natural join Reserves natural join Boat
where rDate = '01-JAN-2025'
and color = 'red';
```

## EXERCISE 3 (CONTD)

For both **Strategy C<sub>2</sub>1** and **Strategy C<sub>2</sub>2**, of the **100 TempC<sub>1</sub>** tuples, **10% \* 100 = 10** tuples are expected to be in the result since **10%** of boats are red.

The join result size  $(11+32-2) = 41$  bytes.

Therefore,  $bf_{\text{joinresult}} = \lfloor 260/41 \rfloor = \lfloor 6.34 \rfloor = 6$ .

Thus, TempC<sub>2</sub> will occupy  $\lceil 10/6 \rceil = \lfloor 1.67 \rfloor = 2$  pages.

Page I/O cost to write TempC<sub>2</sub>: 2

Total page I/O cost for Strategy C<sub>2</sub>1:  $5 + 125 + 2 = 132$

Total page I/O cost for Strategy C<sub>2</sub>: 5 + 200 + 2 = 207

*bf* <sub>Sailor</sub>: 10 tuples/page  
*bf* <sub>Reserves</sub>: 23 tuples/page  
*bf* <sub>Boat</sub>: 8 tuples/page  
*B* <sub>Sailor</sub>: 1,000 pages  
*B* <sub>Reserves</sub>: 4,348 pages  
*B* <sub>Boat</sub>: 125 pages  
 Buffer pages *M*: 100  
 Page size: 260  
 Hash index: Sailor.sailorId  
 B+-tree index: Reserves.rDate  
 (2 levels)  
 Hash index: Boat.boatId  
 Unique Reserves.rDates: 1000  
 Red boats: 10% (100 tuples)  
 Reservations/sailor: 10

```

select *
from Sailor natural join Reserves natural join Boat
where rDate = '01-JAN-2025'
and color = 'red';

```

## EXERCISE 3 (CONT'D)

$bf_{Sailor}$ : 10 tuples/page  
 $bf_{Reserves}$ : 23 tuples/page  
 $bf_{Boat}$ : 8 tuples/page  
 $B_{Sailor}$ : 1,000 pages  
 $B_{Reserves}$ : 4,348 pages  
 $B_{Boat}$ : 125 pages  
 Buffer pages  $M$ : 100  
 Page size: 260  
 Hash index: Sailor.sailorId  
 B<sup>+</sup>-tree index: Reserves.rDate  
                     (2 levels)  
 Hash index: Boat.boatId  
 Unique Reserves rDates: 1000  
 Red boats: 10% (100 tuples)  
 Reservations/sailor: 10

### $C_3$ : Cost of Sailor JOIN Temp $C_2$

Strategy  $C_31$ : block nested-loop join

Read 2 pages of Temp $C_2$  containing 10 tuples which fit in the buffer.

Page I/O cost to read Temp $C_2$ : 2

Read 1,000 pages of Sailor and join with Temp $C_2$  in the buffer.

Page I/O cost to read Sailor: 1,000

Total page I/O cost for Strategy  $C_31$ :  $2 + 1000 = 1,002$

**Best  
overall  
strategy**

Strategy  $C_32$ : indexed nested-loop join using hash index on Sailor.sailorId

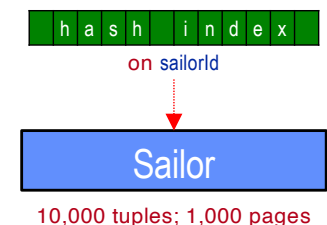
Read 2 pages of Temp $C_2$  containing 10 tuples which fit in the buffer.

Page I/O cost to read Temp $C_2$ : 2

For each of the 10 Temp $C_2$  tuples, use the Sailor hash index on sailorId to retrieve the corresponding Sailor tuple. The cost is 2 page I/Os (1 to the hash index and 1 to access the Sailor tuple) for each Temp $C_2$  tuple.

Page I/O cost to use hash index on Sailor:  $2 * 10 = 20$

Total page I/O cost for Strategy  $C_32$ :  $2 + 20 = 22$



## EXERCISE 3 (CONTD)

### Minimum query processing page I/O cost

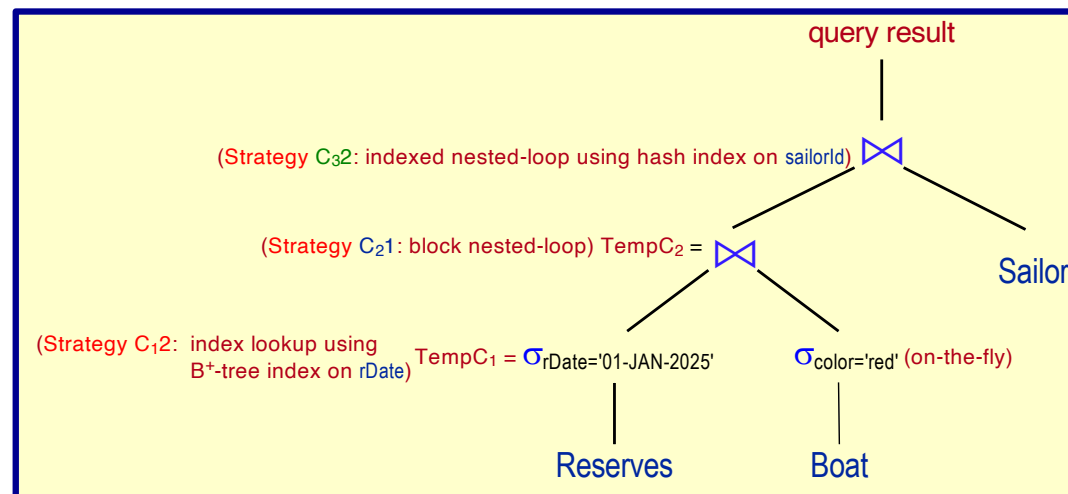
|                              |   |            |
|------------------------------|---|------------|
| $C_12$ : index lookup        | = | 12         |
| $C_21$ : block nested-loop   | = | 132        |
| $C_32$ : indexed nested-loop | = | 22         |
| Total                        |   | <u>166</u> |

### Alternate query processing page I/O cost

|                              |   |            |
|------------------------------|---|------------|
| $C_12$ : index lookup        | = | 12         |
| $C_22$ : hash index          | = | 207        |
| $C_32$ : indexed nested-loop | = | 22         |
| Total                        |   | <u>241</u> |

**Recall:** The cost to write the query result is never included in the cost estimate.

### Minimum Query Processing Page I/O Cost Query Evaluation Tree



## EXERCISE 4

Employee(empId: 4 bytes, name: 35 bytes, title: 2 bytes, salary: 5 bytes, deptId: 4 bytes)

Employee: 50 bytes/tuple; 20,000 tuples; 250 pages

Department(deptId: 4 bytes, deptName: 25 bytes, location: 7 bytes)

Department: 36 bytes/tuple; 100 tuples; 1 page

DeptProject(deptId: 4 bytes, projectId: 4 bytes)

DeptProject: 8 bytes/tuple; 4,000 tuples; 8 pages

Project(projectId: 4 bytes, projectTitle: 20 bytes, budget: 6 bytes, report: 970 bytes)

Project: 1,000 bytes/tuple; 2,000 tuples; 500 pages

Employee salaries: uniformly distributed in the range 10,000 to 110,000.

Project budgets: distributed in the range 10,000 to 30,000 according to the histogram.

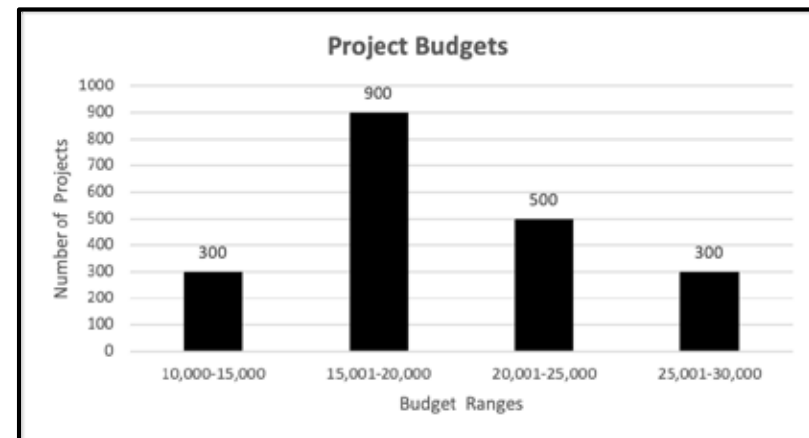
All foreign keys are not null.

Page size: 4,000 bytes Buffer pages: 12

There is a clustering B<sup>+</sup>-tree index  
with 3 levels on salary for Employee.

There is a hash index on deptId for  
Department; Department is ordered on deptId.

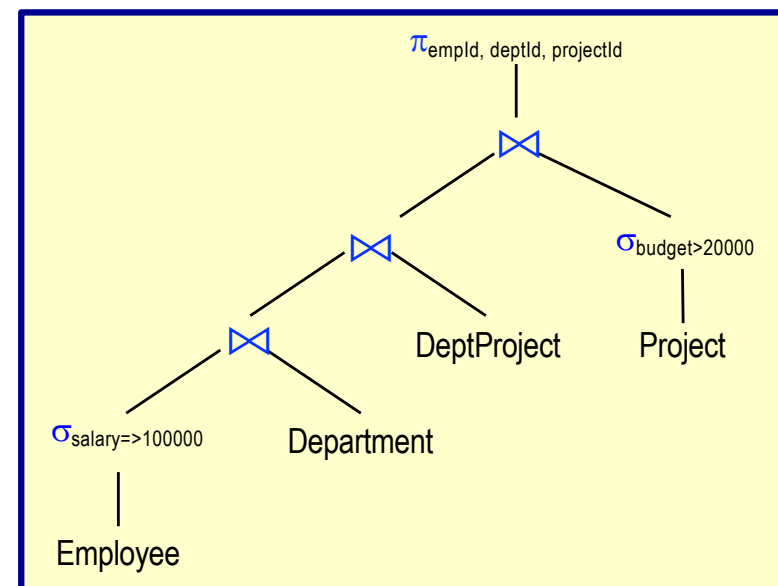
There is a hash index on projectId  
for Project; Project is ordered on projectId.



## EXERCISE 4 (CONT'D)

- a) Estimate the result size of the query in tuples and in pages for the given relational algebra query evaluation tree.
- b) Estimate the query processing page I/O cost using the relational algebra query evaluation tree and the steps given below. The goal is to minimize the total number of page I/Os needed to process the query. Reorder operations, as necessary, to reduce the page I/O cost. Where possible, use pipelining rather than materialization (i.e., keep intermediate results in the buffer where possible). Assume the file organizations and indexes described above. For each step keep only the attributes necessary to process the query.

```
select distinct empld, deptld, projectld
from Employee natural join Department
      natural join DeptProject
      natural join Project
where salary=>100000
and budget>20000;
```





```

select distinct empId, deptId, projectId
from Employee natural join Department
      natural join DeptProject
      natural join Project
where salary=>100000
and budget>20000;

```

## EXERCISE 4 (CONT'D)

```

Employee(empId: 4 bytes, name, title, salary, deptId)
Department(deptId: 4 bytes, deptName, location)
DeptProject(deptId: 4 bytes, projectId: 4 bytes)
Project(projectId: 4 bytes, projectTitle, budget, report)

```

```

nEmployee: 20,000 tuples
BEmployee: 250 pages
nDepartment: 100 tuples
BDepartment: 1 pages
nDeptProject: 4,000 tuples
BDeptProject: 8 pages
nProject: 1,000 tuples
BProject: 500 pages
Page size: 4,000 bytes
Buffer size M: 12 pages

```

a) Estimate the result size of the query in tuples and in pages.

For  $\sigma_{\text{salary} \geq 100000} \text{Employee} \Rightarrow \text{Result A}$  (retain only (empId, deptId) 8 bytes)

$$\text{Selectivity: } \frac{\max(A,r) - v}{\max(A,r) - \min(A,r)} = \frac{110,000 - 100,000}{110,000 - 10,000} = \frac{1}{10}$$

$$\text{Estimated size (tuples): } 20,000 * \frac{1}{10} = \underline{2,000}$$

$$\text{Estimated size (pages): } \lceil 2,000 / \lceil 4000 / 8 \rceil \rceil = \underline{4}$$

For  $\text{Result A} \bowtie \text{Department} \Rightarrow \text{Result B}$  (join is on foreign key of Employee in Result A)

Since deptId in Employee is a not null foreign key referencing Department, every tuple of Result A joins with exactly one tuple of Department. Therefore, the size of Result B is the same as that of Result A. Retain only (empId, deptId) 8 bytes.

$$\text{Estimated size (tuples): } \underline{2,000}$$

$$\text{Estimated size (pages): } \underline{4}$$

For  $\text{Result B} \bowtie \text{DeptProject} \Rightarrow \text{Result C}$  (deptId is not a key in Result B or DeptProject)

Since every Department tuple is related to  $4,000 / 100 = 40$  DeptProject tuples on average, every Result B tuple will join with 40 DeptProject tuples. Retain only (empId, deptId, projectId) 12 bytes.

$$\begin{aligned} \text{Estimated size (tuples): } & n_{\text{Result B}} * n_{\text{DeptProject}} / V(\text{deptId, Department}) \\ & = 2,000 * 4,000 / 100 = \underline{80,000} \end{aligned}$$

$$\text{Estimated size (pages): } \lceil 80,000 / \lceil 4000 / 12 \rceil \rceil = \underline{241}$$

```

select distinct empld, deptld, projectld
from Employee natural join Department
      natural join DeptProject
      natural join Project
where salary=>100000
and budget>20000;

```

## EXERCISE 4 (CONT'D)

Employee(empld: 4 bytes, name, title, salary, deptld)  
 Department(deptld: 4 bytes, deptName, location)  
 DeptProject(deptld: 4 bytes, projectld: 4 bytes)  
 Project(projectld: 4 bytes, projectTitle, budget, report)

$n_{\text{Employee}}$ : 20,000 tuples  
 $B_{\text{Employee}}$ : 250 pages  
 $n_{\text{Department}}$ : 100 tuples  
 $B_{\text{Department}}$ : 1 pages  
 $n_{\text{DeptProject}}$ : 4,000 tuples  
 $B_{\text{DeptProject}}$ : 8 pages  
 $n_{\text{Project}}$ : 1,000 tuples  
 $B_{\text{Project}}$ : 500 pages  
 Page size: 4,000 bytes  
 Buffer size  $M$ : 12 pages

For  $\sigma_{\text{budget}>20000}\text{Project} \Rightarrow \text{Result D}$  (budgets distributed as in histogram)

Selectivity:  $500 + 300 / 2000 = 0.4$

For  $\text{Result C} \bowtie \text{Result D}$  (join is on foreign key of DeptProject in Result C)

Every tuple of Result C will join with a tuple of Result D (i.e., with a Project tuple), but only 0.4 of these tuples will meet the condition

$\sigma_{\text{budget}>20000}$ .

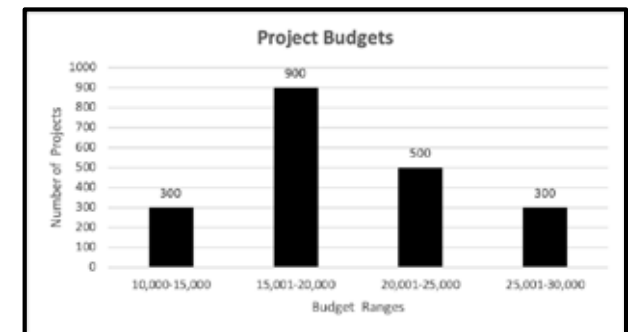
Estimated size:  $n_{\text{Result C}} * 0.4 = 80,000 * 0.4 = 32,000$

Estimated size (pages):  $\lceil 32,000 / \lfloor 4000 / 12 \rfloor \rceil = 97$

Since every employee is related to a unique department and every department is related to a unique set of projects, the combination of (empld, deptld, projectld) in the query result is unique (i.e., there are no duplicates).

Estimated query result size (tuples): 32,000

Estimated query result size (pages): 97



```

select distinct empld, deptld, projectld
from Employee natural join Department
      natural join DeptProject
      natural join Project
where salary=>100000
and budget>20000;

```

## EXERCISE 4 (CONT'D)

$n_{\text{Employee}}$ : 20,000 tuples  
 $B_{\text{Employee}}$ : 250 pages  
 $n_{\text{Department}}$ : 100 tuples  
 $B_{\text{Department}}$ : 1 pages  
 $n_{\text{DeptProject}}$ : 4,000 tuples  
 $B_{\text{DeptProject}}$ : 8 pages  
 $n_{\text{Project}}$ : 1,000 tuples  
 $B_{\text{Project}}$ : 500 pages  
 Page size: 4,000 bytes  
 Buffer size  $M$ : 12 pages  
 B<sup>+</sup>-tree: Employee.salary  
 Hash index: Department.deptld  
 Hash index: Project.projectld

b) Estimate the query processing page I/O cost.

Step 1:  $\sigma_{\text{salary} \geq 100000} \text{Employee} \Rightarrow \text{Result A}$

Strategy 1: linear search on Employee

Strategy 1 page I/O cost: 250

Strategy 2: index lookup using B<sup>+</sup>-tree on salary

Use the B<sup>+</sup>-tree to find the first page where the salary equals 100,000. The Employee relation occupies 250 pages. Since the relation is ordered on salary and from a) it is known that 1/10 of employees have a salary  $\geq 100,000$ , then  $250 * 0.1 = 25$  pages contain Employee records where the salary is  $\geq 100,000$ .

Therefore, the cost is 3 pages I/Os (to search the B<sup>+</sup>-tree) plus 25 page I/Os to retrieve all the qualifying Employee records.

Strategy 2 page I/O cost:  $3 + 25 = 28$

Step 1 minimum page I/O cost: 28

From a) 2,000 tuples are expected to be retrieved. Only empld and deptld are retained which will occupy  $\lceil 2000 / \lfloor 4000 / 8 \rfloor \rceil = 4$  pages and which are kept in the buffer for the subsequent join.

Step 1 result size (tuples): 2,000

Step 1 result size (pages): 4

```

select distinct empId, deptId, projectId
from Employee natural join Department
      natural join DeptProject
      natural join Project
where salary > 100000
and budget > 20000;

```

## EXERCISE 4 (CONT'D)

$n_{Employee}$ : 20,000 tuples  
 $B_{Employee}$ : 250 pages  
 $n_{Department}$ : 100 tuples  
 $B_{Department}$ : 1 pages  
 $n_{DeptProject}$ : 4,000 tuples  
 $B_{DeptProject}$ : 8 pages  
 $n_{Project}$ : 1,000 tuples  
 $B_{Project}$ : 500 pages  
 Page size: 4,000 bytes  
 Buffer size  $M$ : 12 pages  
 B<sup>+</sup>-tree: Employee.salary  
 Hash index: Department.deptId  
 Hash index: Project.projectId

Step 2: Result A ⋈ Department ⇒ Result B

Strategy 1: block nested-loop join

Read the Department relation (1 page) into the buffer and join with the 2,000 tuples from the previous step.

Strategy 1 page I/O cost: 1

Strategy 2: indexed nested-loop join using deptId hash index

For each of the 2,000 Step 1 tuples, use the deptId hash index on Department to join with the matching Department tuple. The cost is 1 page I/O to the index and 1 page I/O to retrieve the Department record for each of the 2,000 tuples.

Strategy 2 page I/O cost:  $2000 * 2 = 4000$

Strategy 3: ignore the join

Since the query result does not contain any attributes from the Department relation other than deptId, which is already in the result of Step 1, and since Result A and DeptProject can be joined directly on deptId, this step can be ignored.

Strategy 3 page I/O cost: 0

Step 2 minimum page I/O cost: 0

Step 2 result size (tuples): 0

Step 2 result size (pages): 0

```

select distinct empld, deptld, projectld
from Employee natural join Department
      natural join DeptProject
      natural join Project
where salary=>100000
and budget>20000;

```

## EXERCISE 4 (CONT'D)

$n_{Employee}$ : 20,000 tuples  
 $B_{Employee}$ : 250 pages  
 $n_{Department}$ : 100 tuples  
 $B_{Department}$ : 1 pages  
 $n_{DeptProject}$ : 4,000 tuples  
 $B_{DeptProject}$ : 8 pages  
 $n_{Project}$ : 1,000 tuples  
 $B_{Project}$ : 500 pages  
 Page size: 4,000 bytes  
 Buffer size  $M$ : 12 pages  
 B+-tree: Employee.salary  
 Hash index: Department.deptld  
 Hash index: Project.projectld

**Step 3:** Result A  $\bowtie$  DeptProject  $\Rightarrow$  Result C

**Strategy:** block nested-loop join

Since there is no index on DeptProject, block nested-loop is the only possible strategy for the join. The cost is that to read DeptProject.

**Strategy page I/O cost: 8**

As estimated in a) each of the 2,000 tuples from Step 1 is expected to match with 40 DeptProject tuples. Thus, the expected result size is 80,000 tuples. Retain only empld, deptld and projectld which will occupy  $\lceil 80,000 / \lfloor 4000 / 12 \rfloor \rceil = 241$  pages. The join result cannot be kept in the buffer but needs to be written to disk requiring 241 page I/Os. (Note: partial results of a step are not retained in the buffer.)

**Join result write page I/O cost: 241**

**Step 3 minimum page I/O cost: 8 + 241 = 249**

**Step 3 result size (tuples): 80,000**

**Step 3 result size (pages): 241**

```

select distinct empld, deptld, projectld
from Employee natural join Department
      natural join DeptProject
      natural join Project
where salary=>100000
and budget>20000;

```

## EXERCISE 4 (CONT'D)

$n_{\text{Employee}}$ : 20,000 tuples  
 $B_{\text{Employee}}$ : 250 pages  
 $n_{\text{Department}}$ : 100 tuples  
 $B_{\text{Department}}$ : 1 pages  
 $n_{\text{DeptProject}}$ : 4,000 tuples  
 $B_{\text{DeptProject}}$ : 8 pages  
 $n_{\text{Project}}$ : 1,000 tuples  
 $B_{\text{Project}}$ : 500 pages  
 Page size: 4,000 bytes  
 Buffer size  $M$ : 12 pages  
 B<sup>+</sup>-tree: Employee.salary  
 Hash index: Department.deptld  
 Hash index: Project.projectld

**Step 4:**  $\sigma_{\text{budget}>20000}\text{Project} \Rightarrow \text{Result D}$

**Strategy 1:** linear search

Read the 500 Project pages and discard tuples that do not meet the condition. From a) we know that 800 tuples will satisfy the condition. We keep only the projectld (4 bytes) for the subsequent join. These 800 tuples will occupy  $\lceil 800 / \lfloor 4000 / 4 \rfloor \rceil = 1$  page which is kept in the buffer.

Strategy 1 page I/O cost: 500

**Strategy 2:** do *on-the-fly* in the buffer during the join in Step 5

Strategy 2 page I/O cost: 0

We could do the selection on Project after the join in Step 5. However, this would increase the join cost as we would need to join the 500 pages of Project and the 241 pages of Result C. Using optimized block nested-loop join in Step 5 would have a page I/O cost of  $\lceil 500 / (12-2) \rceil * 241 + 500 = 12,550$ . Using indexed nested-loop join using the projectld hash index would have a cost of  $80,000 * 2 = 160,000$ . When compared to the join cost if Strategy 1 is used in this step, we conclude that using Strategy 1 is the better choice.

**Step 4 minimum page I/O cost:** 500

**Step 4 result size (tuples):** 800

**Step 4 result size (pages):** 1



```

select distinct empld, deptId, projectId
from Employee natural join Department
      natural join DeptProject
      natural join Project
where salary >= 100000
and budget > 20000;

```

## EXERCISE 4 (CONT'D)

$n_{Employee}$ : 20,000 tuples  
 $B_{Employee}$ : 250 pages  
 $n_{Department}$ : 100 tuples  
 $B_{Department}$ : 1 pages  
 $n_{DeptProject}$ : 4,000 tuples  
 $B_{DeptProject}$ : 8 pages  
 $n_{Project}$ : 1,000 tuples  
 $B_{Project}$ : 500 pages  
 Page size: 4,000 bytes  
 Buffer size  $M$ : 12 pages  
 B+-tree: Employee.salary  
 Hash index: Department.deptId  
 Hash index: Project.projectId

### Step 5: Result C ⋈ Result D

**Strategy:** block nested-loop join

Read the 241 pages of **result C** and join with the 1 page of **Result D** which is in the buffer.

Strategy page I/O cost: 241

Step 5 minimum page I/O cost: 241

Not every **Result C** tuple will join with a **Result D** tuple since 60% of projects were eliminated by the selection in **Step 4**. Therefore, of the 80,000 tuples in **Result C**, we expect only 40% of them to join with a **Result D** tuple. Thus, the estimated result size is  $80,000 * 0.4 = 32,000$  tuples or  $\lceil 32,000 / \lfloor 4000 / 12 \rfloor \rceil = 97$  pages.

Step 5 result size (tuples): 32,000

Step 5 result size (pages): 97

Query processing page I/O cost: 28 + 0 + 249 + 500 + 241 = 1,018

Query result size (pages): 97

**Recall:** The cost to write the query result is never included in the cost estimate.